Assignment Day 9
DSA

Name : **Abhinay Kumar**
PRN : **260240120005**
Form No : 251201864
Course : PGCP - AC
Batch : February 2026

Assignment
1. Implement a hash table class in which each bucket is implemented as a linked list. Use the linked list class that is implemented earlier in module for this implementation.

2: Implement printBFS() method in AdjListGraph class (code written in class today).
The method should traverse graph using BFS and print the vertices.
Graph data will be stored in an adjacency list.

3: Implement printDFS() method in AdjacencyMatrix class (code written in class today).
The method should traverse graph using DFS and print the vertices.
Graph data will be stored in an adjacency matrix.

LinkedListBucket :-

```
package com.dsa.utils;
class LinkedListBucket {
  private LinkedList<String> list = new LinkedList<>();
  public void insert(int key, int value) {

    remove(key);

    String data = key + ":" + value;
    list.insertAtEnd(data);
  }
  public Integer get(int key) {
    Node<String> current = list.head;

    while (current != null) {
      String str = current.getData();
      String[] parts = str.split(":");
      int storedKey = Integer.parseInt(parts[0]);

      if (storedKey == key) {
        return Integer.parseInt(parts[1]);
      }
      current = current.getNext();
    }
    return null;
  }
  public boolean remove(int key) {
```

```
        Node<String> current = list.head;
        Node<String> prev = null;
        int position = 0;
        while (current != null) {
            String str = current.getData();
            String[] parts = str.split(":");
            int storedKey = Integer.parseInt(parts[0]);
            if (storedKey == key) {
                if (position == 0) {
                    list.deleteAtFront();
                } else {
                    list.deleteAtIndex(position);
                }
                return true;
            }

            prev = current;
            current = current.getNext();
            position++;
        }
        return false;
    }
    public void printBucket() {
        if (list.isEmpty()) {
            System.out.println("null");
            return;
        }
        Node<String> current = list.head;
        while (current != null) {
            System.out.print(current.getData() + " -> ");
            current = current.getNext();
        }
        System.out.println("null");
    }
}
```

HashTableClass :-

```
package com.dsa.utils;
public class HashTableClass {
    private LinkedListBucket[] table;
    private int capacity;
    private int size;
    public HashTableClass(int capacity) {
        this.capacity = capacity;
        this.table = new LinkedListBucket[capacity];
        this.size = 0;
        for (int i = 0; i < capacity; i++) {
            table[i] = new LinkedListBucket();
        }
    }
    private int hash(int key) {
```

```
        return Math.abs(key) % capacity;
    }
    public void put(int key, int value) {
        int index = hash(key);
        table[index].insert(key, value);
        size++;
    }
    public Integer get(int key) {
        int index = hash(key);
        return table[index].get(key);
    }
    public boolean remove(int key) {
        int index = hash(key);
        boolean removed = table[index].remove(key);
        if (removed) {
            size--;
        }
        return removed;
    }
    public void printHashTable() {
        System.out.println("Hash Table Contents (Separate Chaining):");
        for (int i = 0; i < capacity; i++) {
            System.out.print("Bucket " + i + ": ");
            table[i].printBucket();
        }
        System.out.println("Total elements: " + size);
    }
    public int size() {
        return size;
    }
}
```

BFSAdjacencyList:-

```
package com.dsa.utils;

import java.util.LinkedList;
import java.util.*;

public class BFSAdjacencyList {
    private int vertices;
    private List<List<Integer>> adjList;

    public BFSAdjacencyList(int vertices) {
        this.vertices = vertices;
        adjList = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
```

```
    adjList.get(u).add(v);
    adjList.get(v).add(u);
}

public void printBFS(int start) {
    boolean[] visited = new boolean[vertices];
    Queue<Integer> queue = new LinkedList<>();

    visited[start] = true;
    queue.add(start);

    System.out.print("BFS Traversal starting from " + start + ": ");

    while (!queue.isEmpty()) {
        int current = queue.poll();
        System.out.print(current + " ");

        for (int neighbor : adjList.get(current)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.add(neighbor);
            }
        }
    }
    System.out.println();
}

public void printBFSAll() {
    boolean[] visited = new boolean[vertices];
    for (int i = 0; i < vertices; i++) {
        if (!visited[i]) {
            printBFSHelper(i, visited);
        }
    }
}

private void printBFSHelper(int start, boolean[] visited) {
    Queue<Integer> queue = new LinkedList<>();
    visited[start] = true;
    queue.add(start);

    while (!queue.isEmpty()) {
        int curr = queue.poll();
        System.out.print(curr + " ");

        for (int neigh : adjList.get(curr)) {
            if (!visited[neigh]) {
                visited[neigh] = true;
                queue.add(neigh);
            }
        }
    }
```

```
    }
}
```

DFSAdjacencyList :-

```
package com.dsa.utils;
public class DFSAdjacencyList {
  private int vertices;
  private int[][] adjMatrix;
  public DFSAdjacencyList(int vertices) {
     this.vertices = vertices;
     adjMatrix = new int[vertices][vertices];
  }
  public void addEdge(int u, int v) {
     adjMatrix[u][v] = 1;
     adjMatrix[v][u] = 1;
  }
  public void printDFS(int start) {
     boolean[] visited = new boolean[vertices];
     System.out.print("DFS Traversal starting from " + start + ": ");
     dfsHelper(start, visited);
     System.out.println();
  }
  private void dfsHelper(int vertex, boolean[] visited) {
     visited[vertex] = true;
     System.out.print(vertex + " ");
     for (int i = 0; i < vertices; i++) {
        if (adjMatrix[vertex][i] == 1 && !visited[i]) {
           dfsHelper(i, visited);
        }
     }
  }
  public void printDFSAll() {
     boolean[] visited = new boolean[vertices];
     for (int i = 0; i < vertices; i++) {
        if (!visited[i]) {
           dfsHelper(i, visited);
        }
     }
     System.out.println();
  }
}
```

Main.java:-

```
package com.dsa.utils;
public class Main {
  public static void main(String[] args) {
     System.out.println("=== Hash Table ===\n");

     HashTableClass ht = new HashTableClass(5);
     ht.put(10, 100);
```

```java
        ht.put(15, 150);
        ht.put(20, 200);
        ht.put(25, 250);
        ht.put(5, 500);
        ht.printHashTable();
        System.out.println("Get 15: " + ht.get(15));
        ht.remove(15);
        System.out.println("\nAfter removing 15:");
        ht.printHashTable();
        System.out.println("\n==============================================");

        System.out.println("1. BFS on Adjacency List Graph:");
        BFSAdjacencyList graph1 = new BFSAdjacencyList(7);


        graph1.addEdge(0, 1);
        graph1.addEdge(0, 2);
        graph1.addEdge(1, 3);
        graph1.addEdge(1, 4);
        graph1.addEdge(2, 5);
        graph1.addEdge(2, 6);
        graph1.printBFS(0);
        System.out.println("\n==============================================");
        System.out.println("2. DFS on Adjacency Matrix Graph:");
        DFSAdjacencyList graph2 = new DFSAdjacencyList(7);
        graph2.addEdge(0, 1);
        graph2.addEdge(0, 2);
        graph2.addEdge(1, 3);
        graph2.addEdge(1, 4);
        graph2.addEdge(2, 5);
        graph2.addEdge(2, 6);
        graph2.printDFS(0);
    }
}
```

Output:-

```
=== Hash Table Demo ===

Hash Table Contents (Separate Chaining):
Bucket 0: 10:100 -> 15:150 -> 20:200 -> 25:250 -> 5:500 -> null
Bucket 1: null
Bucket 2: null
Bucket 3: null
Bucket 4: null
Total elements: 5
Get 15: 150

After removing 15:
Hash Table Contents (Separate Chaining):
Bucket 0: 10:100 -> 20:200 -> 25:250 -> 5:500 -> null
Bucket 1: null
Bucket 2: null
Bucket 3: null
Bucket 4: null
Total elements: 4

==============================================
1. BFS on Adjacency List Graph:
BFS Traversal starting from 0: 0 1 2 3 4 5 6

==============================================
2. DFS on Adjacency Matrix Graph:
DFS Traversal starting from 0: 0 1 3 4 2 5 6
```